Slides for P3639R0 — The `_BitInt` Debate

Document number:
P3721R0

Date:
2025-06-04

Audience:
SG22

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-To:
Jan Schultke <janschultke@gmail.com>

Source:
github.com/Eisenwave/cpp-proposals/blob/master/src/bitint-debate-slides.cow

---

> **IMPORTANT:** This document has custom controls:
>
> - →, ↓: go to the next slide
> - ←, ↑: go to previous slide

The `_BitInt` Debate

P3639R0

Introduction

C23 now has `_BitInt` type for N-bit integers (WG14 [N2763], [N2775]):

```
// 8-bit unsigned integer initialized with value 255.
// The literal suffix wb is unnecessary in this case.
unsigned _BitInt(8) x = 0xFFwb;
```

- would be very useful in C++ for > 64-bit computation
- *needs* to be in C++ to call C functions portably!
- efforts to standardize in C++ abandoned (mainly [N1744], [P1889R1] (TS))
- implemented by GCC and Clang; max: `_BitInt(8'388'608)`
- **Big question**: fundamental type or class type in C++?

Possible implementations

𝔽 – Fundamental type	𝕃 – Library type

`template <size_t N>`
`using bit_int_t =`
`  _BitInt(N);`
 	
`template <size_t N>`
`using bit_uint_t =`
`  unsigned _BitInt(N);`
	`template <size_t N>`
	`class bit_int {`
	`  private:`
	`  _BitInt(N) _M_value;`
	`  public:`
	`  // ...`
	`};`
	`template <size_t N>`
	`class bit_uint`
	`  { /* ... */; };`

𝔽 – Fundamentals can do more

A fundamental type is needed for full C compatibility:

```
switch (_BitInt(32) x = 0uwb) { /* ... */ }
struct S
  { _BitInt(32) x : 10; /* OK, _BitInt(32) bit-field */ };
```

A fundamental type could have *deduction superpowers*:

```
template <size_t N> void f(bit_int_t<N>);
foo(0); // OK, calls foo<32> on most platforms
```

This could also be addressed by P2998, "Deducing function parameter types using alias template CTAD".

𝔽 – Why not reinvent integers?

Users have many grievances with the standard integers:

```
decltype(+uint8_t{1})          // int?!
unsigned(0) < -1               // true?!
uint8_t x = 1000;              // OK?!
int x = 0.5f;                  // OK?!
0 + true                       // OK?!
uint32_t x;                    // optional?!
uint128_t x;                   // why not?!
sizeof(int) == sizeof(long)    // true?!
```

- `_BitInt` solves many issues in C, but not all (e.g. mixed-sign comp.)
- if fundamental in C++, could revamp integer types

𝔽 – Problems with a common spelling

- common spelling useful for C/C++-interoperable headers
- C uses the `_BitInt(N)` and `unsigned _BitInt(N)`
- C++ compatibility macro like `_Atomic(...)` does *not* work

```
#define _BitInt(...) std::bit_int<(__VA_ARGS__)>
unsigned _BitInt(32) x; // error: applying "unsigned" to class type
```

⇒ Both C and C++ developers would need a macro for unsigned types.

```
/* C */   #define _BitUint(...) unsigned _BitInt(__VA_ARGS__)
/* C++ */ #define _BitUint(...) std::bit_uint<(__VA_ARGS__)>
```

𝕃 – Class is easier to implement, teach

```
template <size_t N> class bit_int {
    unsigned long long limbs[/* ... */];
  public:
    /* constructors, operator overloads, etc. */
};
```

- P3140 `std::int_least128_t` got push-back for freestanding
- MSVC does not even support 128-bit yet; no `_BitInt` implementation
- no compiler builtin required, no platform ABI changes
- many existing implementations (e.g. Boost.Multiprecision)
- conversion, overload resolution more easily explained

𝕃 – Fundamentals have *huge* blast radius

- new family of integer types
- countless uses of "integer" in wording revised
- integral promotion/conversion rules, template deduction rules
- library support is tough if bit-precise integers widely supported:
  - N-bit `<numeric>`, `<bit>`, `<simd>`, `<linalg>`
  - N-bit `<charconv>`, `<format>`
  - [...]
- library types could have more limited/gradual adoption

Summary

Key arguments (see [P3639R0] for more):

- 𝔽 – Fundamentals can do more
- 𝔽 – Why not reinvent integers?
- 𝔽 – Problems with a common spelling
- 𝕃 – Class is easier to implement, teach
- 𝕃 – Fundamentals have *huge* blast radius

Author position: neutral / slightly 𝔽

References

[P3639R0] *Jan Schultke.* The _BitInt Debate (2025-02-20)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3639r0.html

[N1744] *Michiel Salters.* Big Integer Library Proposal for C++0x (2005-01-13)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2005/n1744.pdf

[P1889R1] *Alexander Zaitsev et al..* C++ Numerics Work In Progress (2019-12-27)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1889r1%2epdf

[N2763] *Aaron Ballman et al..* Adding a Fundamental Type for N-bit integers (2021-06-21)
https://open-std.org/JTC1/SC22/WG14/www/docs/n2763.pdf

[N2775] *Aaron Ballman, Melanie Blower.* Literal suffixes for bit-precise integers (2021-07-13)
https://open-std.org/JTC1/SC22/WG14/www/docs/n2775.pdf